Memoization

zation · Cache coin-change DP / tax-rate lookups with call-count trac · Build with

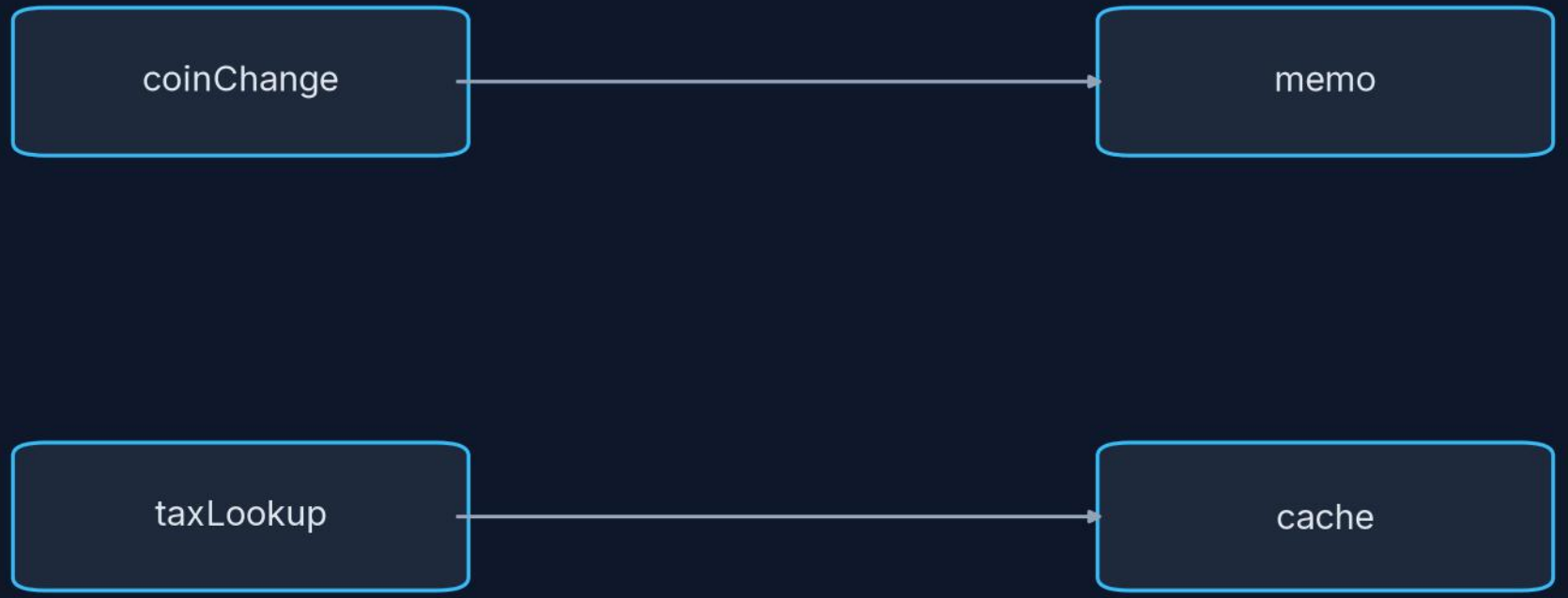
Today's Hook

- Today's force: Cache coin-change DP / tax-rate lookups with call-count trace
- We earn Memoization by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: CoinChangeMemo, Counter

The Problem

- Coin-change ways(amount, coins) — exponential naive vs memoized DP.
- Also memoize expensive tax-rate lookup by region.
- Trace: call counts + elapsed ms printed by ProblemDemo.
- Success: memo ways == naive ways with far fewer calls; tax computeCalls=2 for CA+NY.

Structure



Memoization: Cache coin-change DP / tax-rate lookups with call-

Real-World Scenario

- Running example: Cache coin-change DP / tax-rate lookups with call-count trace
- Key types: CoinChangeMemo, Counter
- Happy-path stdout cue: naive ways=18 calls=961 ms=0
- Failure-path stdout cue: Failure path is asserted in ProblemTest (injected fault / invalid input)

Naive Code

```java
package com.buildwithwaqar.patterns.functional.memoization.problem;

public final class ProblemDemo {
    public static void main(String[] args) {
        System.out.println("=== Memoization: coin-change DP + tax lookup ===");
        int[] coins = {1, 5, 10, 25};
        int amount = 30;

        var naiveC = new CoinChangeMemo.Counter();
        long t0 = System.nanoTime();
        int naive = CoinChangeMemo.naiveWays(amount, coins, naiveC);
        long naiveMs = (System.nanoTime() - t0) / 1_000_000;
        System.out.println("naive ways=" + naive + " calls=" + naiveC.get() + " ms=" + naiveMs);

        var memoC = new CoinChangeMemo.Counter();
        t0 = System.nanoTime();
        int memo = CoinChangeMemo.memoWays(amount, coins, memoC);
        long memoMs = (System.nanoTime() - t0) / 1_000_000;
        System.out.println("memo  ways=" + memo + " calls=" + memoC.get() + " ms=" + memoMs);

        System.out.println("--- tax-rate lookup memoize ---");
        var taxC = new CoinChangeMemo.Counter();
        var tax = CoinChangeMemo.memoize(CoinChangeMemo::expensiveTaxRate, taxC);
        long t1 = System.nanoTime();
        System.out.println("CA1=" + tax.apply("CA") + " CA2=" + tax.apply("CA") + " NY=" + tax.apply("NY"));
        System.out.println("computeCalls=" + taxC.get() + " elapsedMs=" + (System.nanoTime() - t1) / 1_000_000);
    }
```

Naive Code (2)

```
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Cache coin-change DP / tax-rate lookups with call-count trace
- Without Memoization, call sites absorb the wrong details.
- Failure mode to show on camera: Failure path is asserted in ProblemTest (injected fault / invalid input)
- New requirements force shotgun edits across the naive code.

Solution

```java
package com.buildwithwaqar.patterns.functional.memoization.problem;

import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.function.Function;

/** Memoized coin-change DP + tax-rate lookup with call-count / timing traces. */
public final class CoinChangeMemo {
    private CoinChangeMemo() {}

    public static final class Counter {
        private final AtomicInteger calls = new AtomicInteger();
        public int inc() { return calls.incrementAndGet(); }
        public int get() { return calls.get(); }
        public void reset() { calls.set(0); }
    }

    /** Naive exponential recursion. */
    public static int naiveWays(int amount, int[] coins, Counter c) {
        c.inc();
        if (amount == 0) return 1;
        if (amount < 0 || coins.length == 0) return 0;
        int with = naiveWays(amount - coins[0], coins, c);
        int without = naiveWays(amount, Arrays.copyOfRange(coins, 1, coins.length), c);
        return with + without;
    }
```

Solution (2)

```java
public static int memoWays(int amount, int[] coins, Counter c) {
    return memoWays(amount, coins, 0, new HashMap<>(), c);
}

private static int memoWays(int amount, int[] coins, int i, Map<String, Integer> cache, Counter c) {
    c.inc();
    if (amount == 0) return 1;
    if (amount < 0 || i >= coins.length) return 0;
    String key = amount + ":" + i;
    Integer hit = cache.get(key);
    if (hit != null) return hit;
    int with = memoWays(amount - coins[i], coins, i, cache, c);
    int without = memoWays(amount, coins, i + 1, cache, c);
    int ans = with + without;
    cache.put(key, ans);
    return ans;
}

/** Generic memoize wrapper for expensive lookups. */
public static <A, B> Function<A, B> memoize(Function<A, B> f, Counter c) {
    Map<A, B> cache = new HashMap<>();
    return a -> {
        if (cache.containsKey(a)) return cache.get(a);
        c.inc();
        B b = f.apply(a);
        cache.put(a, b);
        return b;
    };
}
```

Solution (3)

```
    public static int expensiveTaxRate(String region) {
        try { Thread.sleep(5); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        return switch (region) {
            case "CA" -> 725; case "NY" -> 800; default -> 500;
        };
    }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.functional.memoization.problem;

public final class ProblemDemo {
    public static void main(String[] args) {
        System.out.println("=== Memoization: coin-change DP + tax lookup ===");
        int[] coins = {1, 5, 10, 25};
        int amount = 30;

        var naiveC = new CoinChangeMemo.Counter();
        long t0 = System.nanoTime();
        int naive = CoinChangeMemo.naiveWays(amount, coins, naiveC);
        long naiveMs = (System.nanoTime() - t0) / 1_000_000;
        System.out.println("naive ways=" + naive + " calls=" + naiveC.get() + " ms=" + naiveMs);

        var memoC = new CoinChangeMemo.Counter();
        t0 = System.nanoTime();
        int memo = CoinChangeMemo.memoWays(amount, coins, memoC);
        long memoMs = (System.nanoTime() - t0) / 1_000_000;
        System.out.println("memo  ways=" + memo + " calls=" + memoC.get() + " ms=" + memoMs);

        System.out.println("--- tax-rate lookup memoize ---");
        var taxC = new CoinChangeMemo.Counter();
        var tax = CoinChangeMemo.memoize(CoinChangeMemo::expensiveTaxRate, taxC);
        long t1 = System.nanoTime();
        System.out.println("CA1=" + tax.apply("CA") + " CA2=" + tax.apply("CA") + " NY=" + tax.apply("NY"));
        System.out.println("computeCalls=" + taxC.get() + " elapsedMs=" + (System.nanoTime() - t1) / 1_000_000);
    }
```

Runnable main (2)

```
}
```

Walkthrough Output

```
=== Memoization: coin-change DP + tax lookup ===
naive ways=18 calls=961 ms=0
memo  ways=18 calls=241 ms=1
--- tax-rate lookup memoize ---
CA1=725 CA2=725 NY=800
computeCalls=2 elapsedMs=12
```

Step-by-Step Trace

- Match stdout to stages — look for: naive ways=18 calls=961 ms=0
- Then the injected failure: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Re-run: java -cp ... 51-memoization ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```
// Java 25 direction for Memoization
// Prefer records + sealed where they shrink the demo.
// Problem under test: Cache coin-change DP / tax-rate lookups with call-count trace
```

Trade-offs

- Use Memoization when «Cache coin-change DP / tax-rate lookups with call-count trace» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Memoization in product language.

Related Patterns

- Name the pattern usually confused with Memoization and separate the forces.
- Sketch the roles for «Cache coin-change DP / tax-rate lookups with call-count trace» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Cache coin-change DP / tax-rate lookups with call-count trace
- Naive pain → Memoization boundary → Demo stdout.
- Remember the failure cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Break a ProblemTest assertion, watch it fail, restore it.